

Projecte: Mini-escena 3D

Grau en Disseny i Desenvolupament de Videojocs
CITM - UPC
Matemàtiques II
Tardor 2025

Enunciat

L'objectiu d'aquest projecte és consolidar la llibreria matemàtica desenvolupada durant el curs aplicant-la en un context gràfic real. Haureu de construir una petita aplicació 3D ("Mini-Scene") capaç de gestionar una jerarquia d'objectes i una càmera virtual.

Partint del codi base proporcionat (que inclou la configuració de finestra amb SDL, context OpenGL i la interfície ImGui), haureu d'implementar la lògica de transformacions locals i globals, el càlcul de matrius de vista (*View Matrix*) i projecció (*Projection Matrix*), i l'enviament d'aquestes dades a la GPU per a renderitzar objectes 3D correctament.

Contingut

El projecte es divideix en quatre blocs principals d'implementació:

1. Estructura de Dades: `GameObject` i `Transform`

Heu de crear una estructura de classes que permeti representar objectes en l'escena amb relacions de jerarquia.

1. **Transform:** Ha d'emmagatzemar la posició, rotació (preferiblement en Euler angles per a la UI, convertits a Quaternions/Matrius internament) i escala.
2. **Mètode `GetLocalMatrix()`:** Ha de retornar la matriu $M_{local} = T \cdot R \cdot S$ utilitzant la vostra funció `Matrix4x4::FromTRS`.
3. **Classe `GameObject`:** Ha de contenir un `Transform`, un punter al pare (`parent`) i una llista de fills.
4. **Mètode `GetGlobalMatrix()`:** Ha de calcular recursivament la matriu de món.

$$M_{global} = M_{parent_global} \cdot M_{local}$$

Si l'objecte no té pare (és l'arrel), la global és igual a la local.

2. Virtual Camera

Heu d'implementar una classe o estructura `Camera` que contingui la seva pròpia transformació i els paràmetres de projecció.

1. **View Matrix:** La matriu de vista és la inversa de la transformació global de la càmera.

$$V = (M_{camera_global})^{-1}$$

Heu d'utilitzar el vostre mètode `Matrix4x4::InverseTR`.

2. **Projection Matrix:** Heu de construir la matriu de projecció en perspectiva a partir dels paràmetres: FOV horitzontal, *Aspect Ratio*, *Near Plane* i *Far Plane*. Cal calcular els plans del frustum (*top, bottom, right, left*) i omplir la matriu corresponent.

3. Renderitzat (Pipeline Gràfic)

Al bucle principal (*main loop*), haureu de configurar el renderitzat de l'escena. Es proporciona un fitxer auxiliar `GraphicsUtils.hpp` per facilitar l'enviament de dades a la GPU, i la malla del model per defecte (`mesh.hpp`).

1. **Abans de dibuixar els objectes (un cop per frame):**

- Obtenir la *View Matrix* (V) i la *Projection Matrix* (P) de la càmera activa.

2. **Per a cada objecte de l'escena:**

- Calcular la *Model Matrix* (M_{global}) de l'objecte actual.
- Utilitzar la funció `GraphicsUtils::UploadMVP(programID, M, V, P)` per enviar les matrius al shader.
- Dibuixar la *mesh* de l'objecte.

3. **Vertex Shader i Fragment Shader (vs.glsl, fs.glsl):**

- Es proporciona un shader bàsic però incomplet. Haureu d'escriure la línia del `gl_Position` multiplicant les matrius amb el vèrtex en l'ordre matemàtic correcta.

4. Interfície d'Usuari (ImGui)

L'aplicació ha de permetre manipular l'escena en temps real:

- **Jerarquia:** Llistar els objectes existents i permetre afegir nous cubs (com a fills de l'objecte seleccionat o a l'arrel).
- **Inspector:** Modificar posició, rotació (Euler) i escala de l'objecte seleccionat.
- **Configuració de càmera:** Modificar la posició/rotació de la càmera i els paràmetres de projecció (FOV, Near, Far).

Criteris d'avaluació

Cada apartat té una puntuació específica sobre 10 punts totals:

• **Jerarquia i transformacions (5 punts):**

- Es poden afegir múltiples objectes a l'escena i crear jerarquies.
- En moure, rotar o escalar un objecte pare, els seus fills l'han de seguir en consonància.
- Transformar un fill no ha d'afectar la *transform* del pare (ni dels germans).

• **Càmera i visualització (5 punts):**

- La càmera es comporta correctament a l'espai 3D (ex: si la càmera es mou cap enrere, els objectes s'han d'allunyar visualment).
- En modificar el FOV a la UI, s'ha d'apreciar un efecte de zoom (ampliació/reducció de la perspectiva) en temps real.
- Els objectes que quedin fora del *frustum* han de desaparèixer.

• **Extres (+2 punts; mitjana amb nota final):**

- Controls per moure i rotar la càmera (*mouse/keys*).
- Aplicar un *focus* amb la càmera: moviment amb LERP i rotació amb SLERP.
- Orbitar al voltant d'un objecte (vigileu el *Gimbal Lock*).
- Altres implementacions matemàtiques rellevants per a un *Game Engine*.

El projecte és **obligatori**. La nota de l'entrega correspon al 15% de la **nota final**.

Si es detecten còpies, la nota final és **zero**.

Si no s'entrega el vídeo, la nota final és **zero**.

Entrega

Per a l'entrega del projecte, podeu fer grups de **màxim 3 persones**. Via ATENEA, seguint el format `Cognoms1Nom1_Cognoms2Nom2_Cognoms3Nom3_Projecte.pdf`, amb el següent contingut:

- **Arxius de codi i executable:**

- Feu una *release* a GitHub amb el projecte llest per compilar i l'executable, abans de la data límit. Només poden sortir com a *contributors* els membres del grup. Adjunteu al document l'enllaç al repositori.

- **Vídeo explicatiu:**

- Pengeu a *YouTube* o a qualsevol altra plataforma similar un vídeo de màxim 5 minuts demostrant la funcionalitat del projecte, abans de la data límit. Adjunteu al document l'enllaç al vídeo.

L'entrega s'avaluarà conjuntament amb els membres del grup.

Data límit d'entrega: 11 de gener de 2026.